

---

# CSE Web Data Collection

Document for 25ada073 - Sasini

## **Your part: Crawling for documents, and cleaning the data**

Everything behind your part of the project - the ideas, the reasons, and the code - explained step by step, from the beginning.

## **DA 2009 - Data Collection Methods II**

Assignment 2 (Group Project)

University of Colombo

25ada072 - Pasindu | 25ada073 - Sasini | 25ada141 - Salaama | 24ada076 - Nuhan

# Contents

|                                           |    |
|-------------------------------------------|----|
| How to use this document                  | 3  |
| The project in simple words               | 4  |
| How the web works                         | 6  |
| The ideas behind your part                | 9  |
| Reading Python                            | 13 |
| Your file: src/crawler.py                 | 19 |
| Your files: the Scrapy spider             | 22 |
| Your file: src/cleaner.py                 | 26 |
| How your part fits into the whole project | 30 |
| Ethics - what every member must know      | 31 |
| Showing your part in the viva             | 32 |
| Practice questions                        | 34 |
| Word list                                 | 41 |

# How to use this document

This document explains your part of the project from the very beginning. It does not assume you have seen any of it before. Each chapter builds on the one before it, so read it in order the first time.

Your files are:

- `src/crawler.py`
- `src/cleaner.py`
- `scrapy_project/cse_crawler/spiders/cse_announcements.py`
- `scrapy_project/cse_crawler/settings.py`

You will see four kinds of coloured box:

**Think of it like this.** a comparison with something from everyday life, to make a new idea easier to picture.

**Remember:** the one thing to take away from a section. If you only read the green boxes, you will still know the main points.

**Be careful:** a trap, or something an examiner might test you on.

**Try it yourself.** something to type on the computer, and what should happen.

Grey boxes in a typewriter font are **code** - real lines copied from our project files, not made-up examples.

## A plan for the week before the viva

| Day     | What to do                                                                                         |
|---------|----------------------------------------------------------------------------------------------------|
| 1       | Read chapters 1 to 3. They explain the project and the ideas behind your part.                     |
| 2       | Read the chapter on reading Python, with your own files open beside it.                            |
| 3 and 4 | Read the chapters on your files, one at a time. Run each file as the 'Try it yourself' boxes show. |
| 5       | Read how your part fits into the whole project, and the ethics chapter.                            |
| 6       | Practise the viva chapter out loud - say your 30-second explanation until it feels natural.        |
| 7       | Go through the practice questions at the end. Cover the answer and try to say it first.            |

# The project in simple words

## What a stock exchange is

A company can split its ownership into many small pieces called **shares**. If you buy one share of a bank, you own a tiny piece of that bank.

A **stock exchange** is the marketplace where people buy and sell those shares. The price of a share goes up when many people want to buy it, and down when many people want to sell it. The **Colombo Stock Exchange** - CSE for short - is Sri Lanka's stock exchange, and its website is [www.cse.lk](http://www.cse.lk).

**Think of it like this.** a stock exchange is like a big weekly fair where, instead of vegetables, people trade small pieces of companies. The website is the notice board at the entrance that shows today's prices.

## What we were asked to do

Our group was given the CSE website and asked to collect data from it with a computer program, using the methods taught in the course, and to do it **ethically** - politely, legally, and without causing the website any trouble.

## What we collect

| What            | What it means                                 | Example                                                 |
|-----------------|-----------------------------------------------|---------------------------------------------------------|
| Share prices    | What every company traded at today            | Sampath Bank: Rs 141.00                                 |
| Company details | Facts about one company                       | Its total value, its highest and lowest price this year |
| Announcements   | Notices companies publish, often as PDF files | 'A dividend will be paid', 'A meeting will be held'     |

After collecting, we tidy the data, check it makes sense, draw charts, and save it as files that other people can open.

## Who did what

| Member                          | Part of the project                                              |
|---------------------------------|------------------------------------------------------------------|
| 24ada076 - Nuhan                | Ethical scraping and robots.txt, BeautifulSoup, and reading PDFs |
| 25ada072 - Pasindu              | Getting the data from the site, and Selenium                     |
| 25ada073 - Sasini (this is you) | Crawling for documents, and cleaning the data                    |
| 25ada141 - Salaama              | OCR, saving the files, and the charts                            |

The app itself (app.py) was put together by all four of us. Each tab in the app is labelled with the member whose part it shows.

## The one surprise that shaped the whole project

The first thing we tried was the ordinary method from the course: download the web page and pick the prices out of it. It found almost nothing - about 24 letters of text, and no tables at all.

The reason is that the CSE website is built with **JavaScript**. The page arrives nearly empty, and a small program inside your browser then fetches the prices and fills them in. A plain download never runs that program, so it never sees the prices.

So we opened the site in a browser, pressed **F12**, and watched the **Network** tab, which lists everything the page asks for. It showed the page fetching its prices from addresses such as `https://www.cse.lk/api/tradeSummary`. We read those addresses directly instead, and one request gives us every company at once.

**Remember:** the CSE page is built by JavaScript, so we read the data from the same addresses the page itself uses. That one decision explains why the project is built the way it is.

# How the web works

Before looking at any code, it helps to understand what actually happens when a computer 'visits' a website. There are only a few ideas, and every part of our project uses them.

## Requests and responses

When you open a web page, your computer sends a message to another computer, called a **server**, asking for that page. This message is a **request**. The server sends something back - the page, a file, or some data. That is the **response**.

**Think of it like this.** writing a letter and getting a reply. You write to an address and ask for something. A little later, a reply arrives in your letterbox. Every single thing a website shows you - each picture, each price - arrives this way, one letter and one reply at a time.

## Addresses - URLs

Every page and file on the web has an address, called a **URL**. For example:

```
https://www.cse.lk/api/tradeSummary
```

| Part              | What it means                                                                                         |
|-------------------|-------------------------------------------------------------------------------------------------------|
| https://          | How to talk to the server. The 's' means the conversation is locked so nobody can read it on the way. |
| www.cse.lk        | Which server - the CSE website.                                                                       |
| /api/tradeSummary | Which thing on that server we want. This part is called the <b>path</b> .                             |

## GET and POST - two ways of asking

There are two common kinds of request. A **GET** request simply asks for something: 'please give me this page'. A **POST** request sends some information along with the question, like filling in a form before handing it over.

Most of the CSE data addresses only answer **POST** requests - if you send a GET, they reply with an error. That is their rule, not ours, and we found it by trying both.

## Status codes - the result stamped on every reply

Every response comes with a three-digit number that says how it went. These were taught in Week 2:

| Code | Meaning           | In everyday words                 |
|------|-------------------|-----------------------------------|
| 200  | OK                | Here is what you asked for.       |
| 301  | Moved Permanently | That has moved - look over there. |
| 403  | Forbidden         | You are not allowed to have this. |
| 404  | Not Found         | There is no such page.            |

| Code | Meaning               | In everyday words            |
|------|-----------------------|------------------------------|
| 500  | Internal Server Error | Something broke on our side. |

## Headers - the envelope

Along with every request travel a few lines of extra information called **headers**. One of them, the **User-Agent**, says what program is asking. A browser sends something like 'Chrome on Windows'. Our project sends its own name, the university, the course, and an email address.

**Think of it like this.** the User-Agent is like writing your name and address on the back of the envelope. The receiver knows who wrote, and can write back if there is a problem.

## HTML - what a web page is made of

A web page is a text file written in **HTML**. It uses **tags** - words inside angle brackets - to label each part of the page:

```
<html>
  <head>
    <title>Trade Summary</title>
  </head>
  <body>
    <h1>Today's prices</h1>
    <table>
      <tr> <th>Company</th> <th>Price</th> </tr>
      <tr> <td>Sampath Bank</td> <td>141.00</td> </tr>
    </table>
  </body>
</html>
```

Tags usually come in pairs: <title> opens and </title> closes. Whatever sits between them is the content. Tags sit inside other tags, like boxes inside boxes: the table is inside the body, each row <tr> is inside the table, and each cell <td> is inside a row.

## JSON - data in a tidy shape

Websites do not only send pages. They also send plain **data**, and the most common format for that is **JSON**. JSON is made of **names and values**, grouped with curly brackets { }, and lists grouped with square brackets [ ]. This is a real piece of what CSE sends us:

```
{
  "symbol": "SAMP.N00000",
  "name": "SAMPATH BANK PLC",
  "price": 141.0,
  "change": 1.0
}
```

**Think of it like this.** JSON is like a neatly filled-in form. Every box has a label ('name', 'price') and an answer. A program can read it straight away, with no need to search through the decorations of a web page.

## Static and dynamic pages

|                              | Static page                          | Dynamic page                               |
|------------------------------|--------------------------------------|--------------------------------------------|
| Where the content is         | Already written inside the HTML file | Added by JavaScript after the page arrives |
| Can a plain download see it? | Yes                                  | No                                         |
| Example                      | A list of university departments     | CSE's live prices                          |

**Think of it like this.** a static page is a printed menu - everything is already on it. A dynamic page is an empty chalkboard; a waiter writes today's specials on it after you sit down. If you take a photo the moment you walk in, you photograph an empty board.

**Try it yourself.** open <https://www.cse.lk/equity/trade-summary> in Chrome or Edge. Right-click and choose **View Page Source** - this shows what the server sent, and you will not find the prices in it. Now right-click a price and choose **Inspect** - this shows the page after JavaScript ran, and the prices are there.

# The ideas behind your part

Your part has two halves. The first is **crawling** - finding documents by following links, and downloading them politely. The second is **cleaning** - tidying the data once it has arrived, because data taken from a website is never ready to use straight away.

## Scraping and crawling are different jobs

**Scraping** means taking data out of a page you already know about. **Crawling** means finding pages by following links from one to the next. The Week 6 lecture compared them:

|         | Web scraping                         | Web crawling                           |
|---------|--------------------------------------|----------------------------------------|
| Purpose | Take specific data out of pages      | Browse and discover pages              |
| Scope   | Particular pages you choose          | Many pages, found by following links   |
| Output  | Tidy data - CSV, JSON                | A list of the pages or files found     |
| Focus   | Getting the data out and cleaning it | Finding links and moving between pages |

Both use a program to visit pages, both read what comes back, and both gather web data. Most of our project is scraping: Pasindu's code knows the exact address of the share prices and goes straight there. **Your part is the crawling.** We do not know in advance which documents CSE has published, so your code reads the list, works out where each document lives, and follows each link.

**Think of it like this.** scraping is going to a shop you know and buying one thing. Crawling is walking through a market, reading every signpost, and following each one to see what is there.

## Why crawlers are called spiders

The lecture explained the nickname: a crawler starts from a few known addresses and follows links outwards, building a network of pages it has found - like a spider moving across its web. Scrapy, the framework you used, calls its crawlers **spiders** for the same reason.

## Finding where each document lives

Pasindu's code asks CSE for the list of circulars - official notices, each with a PDF attached. Each record in the list looks like this (shortened):

```
{
  "id": 14364,
  "fileText": "DE-LISTING OF THE ORDINARY VOTING SHARES OF ...",
  "uploadedDate": "06 Aug 2026 09:21:30 AM",
  "path": "upload_report_file/hHXxBdUT5ylLsSvz_6Aug2026...pdf"
}
```

The path is only **part** of an address. On its own it goes nowhere. It is like being told 'flat 4' without the street name. We needed the rest.

The PDFs are not kept on the website itself. Big files are often stored on a separate server built just for delivering files, called a **CDN** (Content Delivery Network). We tried the likely addresses until one gave back a real PDF:

| Address we tried                            | What came back          |
|---------------------------------------------|-------------------------|
| <code>https://www.cse.lk/cmt/ + path</code> | 404 - no such page      |
| <code>https://cdn.cse.lk/ + path</code>     | 403 - not allowed       |
| <code>https://cdn.cse.lk/cmt/ + path</code> | <b>200 - a real PDF</b> |

So every document's full address is `https://cdn.cse.lk/cmt/` followed by its path.

## How do you know a file is really a PDF?

Every file is really just a long row of **bytes** - small numbers from 0 to 255. Many file types start with a few fixed bytes that announce what they are, a bit like a label on the front of a box. Every PDF begins with the four characters %PDF.

So before saving anything, your crawler looks at the first four bytes. If a server sent back an error page instead of a document, it would start with something like `<htm` - and we would not save it as a PDF.

**Think of it like this.** checking the stamp on an envelope before opening it. If it does not say 'PDF', whatever is inside is not what you asked for.

## Crawling politely

A crawler can do a lot of damage quickly, because following links can go on forever. Yours has five limits:

1. **A download limit** - only the first few documents, not every one on the site. The lecture: 'only scrape the data you need'.
2. **No duplicates** - the same document can appear twice in the list; it is only downloaded once.
3. **Skip what we already have** - if a PDF is already saved, it is not downloaded again.
4. **One at a time, with a pause** - every download goes through Nuhan's `fetch()`, so it waits 1.5 seconds and follows `robots.txt`.
5. **Stay on CSE** - the Scrapy spider is told it may only visit `cse.lk` addresses.

## Scrapy, the crawling framework

**Scrapy** is a Python framework built specially for crawling. A **framework** is bigger than a library: instead of you calling its tools, it runs the show and calls **your** code at the right moments. The lecture described it as fast, open source, and widely used in industry.

How Scrapy works, from the Week 6 lecture:

1. **Start a project** - `scrapy startproject name` creates the folders.
2. **Write a spider** - a class that says where to start and what to do with each page.
3. **The engine sends the requests** - Scrapy queues them up and sends them, following its settings.
4. **parse() takes the data out** - Scrapy hands each reply to your code.
5. **Store the results** - to a JSON or CSV file.
6. **Run it** - `scrapy crawl spider_name`.

**Think of it like this.** Scrapy is like a delivery company. You tell it where to start and what to do with each parcel. It handles the vans, the timetable, the queue and the paperwork - you only write the part that

opens the parcels.

| Scrapy word | Meaning                                                                     |
|-------------|-----------------------------------------------------------------------------|
| Spider      | Your class - where to start, and what to do with each reply                 |
| Request     | One page or file to fetch                                                   |
| Callback    | The function Scrapy calls when that reply arrives                           |
| yield       | Hand something back to Scrapy - a new request to follow, or a finished item |
| Item        | One finished record, with named fields                                      |
| Settings    | The rules - delays, robots.txt, how many at once                            |

## Why data needs cleaning

The Week 1 lecture listed a common mistake: 'data collected through scraping is ready for analysis'. It is not. Data arrives in whatever shape the website sent it. Before anyone can sort it, total it or chart it, it has to be tidied.

A computer stores each value as a particular **type**, and the type decides what you can do with it:

| Type    | Example          | What you can do                        |
|---------|------------------|----------------------------------------|
| Number  | 141.0            | Add, average, sort by size             |
| Text    | "141.0"          | Search it, join it - but not add it up |
| Date    | 2026-09-25 14:39 | Sort by time, find the day or month    |
| Missing | None / NaN       | Nothing - it marks a gap               |

We found four problems in the CSE data:

| Problem                  | Example                      | Why it matters                                                      |
|--------------------------|------------------------------|---------------------------------------------------------------------|
| Dates as huge numbers    | 1790326797431                | pandas thinks it is an ordinary number - you could even average it  |
| Numbers stored as text   | "141.0"                      | Text sorts by its first character, so '9' comes after '100'         |
| Completely empty columns | a column of nothing but gaps | Clutters every table and every saved file                           |
| Extra spaces             | " SAMPATH BANK "             | Looks the same as 'SAMPATH BANK' to a person, but not to a computer |

**Think of it like this.** cleaning data is like tidying a kitchen before cooking: putting things in the right cupboards, labelling the jars, throwing out empty boxes. Nothing new is added - it just becomes usable.

## Cleaning and checking are different jobs

**Cleaning** fixes the shape of the data - the types, the empty columns, the repeated rows. **Checking** asks whether the values make sense.

A share price of minus five rupees would be perfectly tidy - a number, in the right column - and completely wrong. Cleaning would never notice. Only a check that says 'prices cannot be negative' would.

**Remember:** crawling finds and downloads the documents politely; cleaning makes the data usable; checking makes sure it is believable. Your part does all three.

# Reading Python

To explain your code, you first need to be able to read it. This chapter explains every piece of Python that appears in your files. Every example is a real line from your own code, so by the end of the chapter you will recognise all of it.

**Try it yourself.** open `src/crawler.py` in VS Code or Notepad and keep it beside this document while you read. Find each example in the file as you go.

## Comments - notes for people

A line that starts with `#` is a **comment**. Python ignores it completely. It is there for people reading the code. Our files are full of comments explaining why each part exists.

From your code:

```
# Every PDF file starts with the characters %PDF. If ours does
# not, we received something else and should not save it.
```

From `download_documents()`. It explains why the next line checks the first four bytes. Python skips both lines.

## Variables - labelled boxes

A **variable** is a name that holds a value. The `=` sign means 'put the thing on the right into the box on the left'. It does not mean 'equals' the way it does in maths.

**Think of it like this.** a variable is a labelled box. `price = 141.0` means: take a box, write 'price' on the label, and put 141.0 inside.

From your code:

```
wanted = documents.head(limit)
```

From `download_documents()`. `documents.head(5)` is the first five rows of the table of documents. They go into a box called `wanted` - the ones we will actually download.

## Text, and filling in blanks with f-strings

Text in Python is written inside quote marks, like `"hello"`. It is called a **string**. A string that starts with `f` is an **f-string**: anything inside curly brackets `{ }` is replaced with the value of that variable.

**Think of it like this.** an f-string is a form letter with blanks. `f"Dear {name}"` fills in the blank with whatever name holds.

From your code:

```
filename = f"{row.get('id', number)}_{title[:60]}"
```

From `download_documents()`. It builds a file name from the document's id number and the first 60 characters of its title, such as `14364_DE-LISTING OF THE ORDINARY VOTING SHARES OF ASSOCIATED MOTOR`. `title[:60]` means 'the first 60 characters' - explained properly below.

## Numbers, and True and False

Python has whole numbers like `30`, numbers with a decimal point like `1.5`, and two special values: `True` and `False`. `None` means 'nothing here yet'.

From your code:

```
DOWNLOAD_DELAY = 1.5
CONCURRENT_REQUESTS = 1
CLOSESPIDER_ITEMCOUNT = 40
```

From the spider's `settings.py`: wait 1.5 seconds, send only one request at a time, and stop after 40 items.

## Lists - things in order

A **list** holds several values in order, inside square brackets. `.append()` adds one more to the end. `[]` on its own is an empty list, ready to be filled.

**Think of it like this.** a list is a numbered shopping list. You can add items to the end, and read them back in order.

From your code:

```
found = []
for _, row in circulars.iterrows():
    found.append({ ... })
```

From `find_documents()`. Start with an empty list and add one entry for each circular. `iterrows()` goes through a table one row at a time.

## Dictionaries - values with labels

A **dictionary** holds values under names, inside curly brackets: `{"name": value}`. You get a value back by its name: `reply["ok"]`. The method `.get(name, default)` does the same, but gives back the default instead of crashing if the name is missing.

**Think of it like this.** a dictionary is a set of labelled drawers. You open a drawer by reading its label, not by counting from the top.

From your code:

```
found.append({
    "Title": str(row.get("fileText", "Untitled")).strip(),
    "Date": row.get("uploadedDate", ""),
    "url": row.get("pdf_url"),
    "id": row.get("id"),
})
```

Each document becomes a small dictionary with four labels. `row.get("fileText", "Untitled")` means 'the title, or the word Untitled if there is none'.

## Functions - recipes

A **function** is a named set of steps. `def` starts one. The names in brackets are its **parameters** - the ingredients it needs. `return` hands back the result. The function does nothing until somebody **calls** it by writing its name followed by brackets.

**Think of it like this.** a function is a recipe. The parameters are the ingredients, the indented lines are the steps, and `return` is the finished dish.

From your code:

```
def download_documents(documents, limit=5, scraper=None,
                      progress=None):
```

The ingredients are the table of documents, how many to download, a scraper and a progress bar. `limit=5` is a **default**: if nobody says how many, it downloads 5. `scraper=None` means 'none given - make one yourself'.

## if and else - making decisions

`if` runs the indented lines only when a condition is true. `else` runs when it is not. `not` turns true into false and the other way round.

From your code:

```
if not content[:4] == b"%PDF":
    results.append({"Title": title[:65],
                  "Result": "Not a PDF file", "File": "-"})
    continue
```

If the first four bytes are not %PDF, note it in the results and move on to the next document without saving anything.

## for loops - doing the same thing for each item

A **for loop** repeats its indented lines once for every item in a list or table. `enumerate(..., start=1)` also counts them for you: 1, 2, 3.

**Think of it like this.** a for loop is a teacher going down the register, doing the same thing for each name.

From your code:

```
for column in tidy.columns:
```

From `clean()`. Go through the table's columns one by one - symbol, name, price, and so on - checking each to see if it holds dates stored as numbers.

## import - taking a toolbox off the shelf

Python comes with some tools built in. For everything else we **import** a library - a toolbox somebody else wrote. `import pandas as pd` means 'load pandas, and let me call it pd for short'. `from X import Y` takes just one tool out of the box.

From your code:

```
import scrapy
import pandas as pd
from src.api_client import CSEApi
from src.ethics import Scraper
```

Your files use Scrapy for the spider, pandas for the tables, Pasindu's CSEApi to get the list of circulars, and Nuhan's Scraper to download politely. The last two are our own files - you can import code from inside the project the same way.

## try and except - a safety net

Code inside `try` is attempted. If anything goes wrong, Python jumps to `except` instead of crashing. We use this wherever the internet or a file might fail.

**Think of it like this.** `try` and `except` is a trapeze artist with a safety net. They attempt the jump; if they fall, the net catches them and the show goes on.

From your code:

```
try:
    reply = json.loads(response.text)
except json.JSONDecodeError:
    self.logger.warning(f"{response.url} did not return JSON")
    return
```

From the spider's `parse()`, `json.loads` turns JSON text into Python lists and dictionaries. If the reply is not valid JSON, the spider notes a warning and stops handling that reply, instead of crashing the whole crawl.

## Classes, objects and self

A **class** is a design for making things that hold both information and actions. The thing you make from the design is an **object**. Functions written inside a class are called **methods**. `__init__` is the special method that runs once, when the object is first made - it sets it up. Inside a class, **self** means 'this particular object'.

**Think of it like this.** a class is a cake tin, and each object is a cake made from it. `self` means 'this cake' - so `self.delay` is this cake's own delay, not somebody else's.

From your code:

```
class CseAnnouncementsSpider(scrapy.Spider):

    name = "cse_announcements"
    allowed_domains = ["cse.lk", "cdn.cse.lk", "www.cse.lk"]
```

The name in brackets, `scrapy.Spider`, means your class is **built on top of** Scrapy's own spider. It gets everything Scrapy's spider can already do, and you only add what is special about yours. This is called **inheritance**. name is what you type after `scrapy crawl`.

## pandas DataFrames - tables in Python

A **DataFrame** is a table with rows and named columns, like a sheet in Excel. pandas is the library that provides it. `pd.DataFrame(rows)` builds one from a list of dictionaries, `df.head(5)` shows the first five rows, and `df["price"]` picks out one column.

From your code:

```
df = df.drop_duplicates(subset=["url"]).reset_index(drop=True)
```

From `find_documents()`. Remove rows with the same address, so no document is downloaded twice. `reset_index(drop=True)` rennumbers the rows 0, 1, 2 afterwards, so there are no gaps where the repeats were.

## Slicing - taking part of a list or text

Square brackets with a colon take a piece of something. The number before the colon is where to start (blank means the beginning), and the number after is where to stop:

```
title[:60]      # the first 60 characters
content[:4]     # the first 4 bytes
table[1:]       # everything except the first item
```

Counting starts from 0, and the stop position itself is not included, so `[ :4]` gives positions 0, 1, 2 and 3 - four things.

## Bytes

Text written with a `b` in front, like `b"%PDF"`, is **bytes** - raw file contents - rather than ordinary text. A downloaded PDF arrives as bytes, so we compare it with bytes.

## continue - skip to the next one

Inside a loop, `continue` means 'stop here for this item and go straight on to the next'. Your crawler uses it to skip a document that is already saved, failed, or is not a PDF.

## any() - is at least one true?

`any(...)` answers True if at least one thing is true. In `clean()` it asks: does this column's name contain 'date', 'time' or 'created'?

```
if not any(word in column.lower()
            for word in ("date", "time", "created")):
    continue
```

`column.lower()` makes the name small letters first, so `lastTradedTime` still matches 'time'.

## lambda - a tiny one-line function

lambda makes a small function without a name, written in one line. It is handy when you need a function only once:

```
tidy[column].apply(  
    lambda value: value.strip() if isinstance(value, str) else value)
```

Read it as: 'for each value: if it is text, trim the spaces off; otherwise leave it exactly as it is'.

## yield - handing things back one at a time

return gives back one answer and the function ends. yield gives back one thing and **keeps going**, ready to give back another. Scrapy spiders use yield to hand Scrapy each new request or finished item as soon as it is ready.

## async def

async def marks a function that can pause and let other work happen while it waits, such as waiting for a reply from a server. Newer versions of Scrapy need the first function to be written this way. You do not need to understand it deeply - just know that Scrapy requires it.

## Running a file

Every file in our project can be run on its own to show what it does. You open a terminal in the project folder, switch on the project's Python, and ask Python to run the file:

```
cd D:\DA2009\CSE-Scraping-Project  
.venv\Scripts\activate  
python -m src.ethics
```

.venv is the project's own copy of Python with all the libraries installed. -m src.ethics means 'run the file ethics.py inside the src folder'. The part at the bottom of each file that starts if \_\_name\_\_ == "\_\_main\_\_": is what runs when you do this.

# Your file: src/crawler.py

Finds the PDF documents CSE has published, and downloads them one at a time. This is the crawler the app uses.

It works in two steps: first find the documents, then fetch them. Keeping the steps apart means we can see the whole list before downloading anything.

## Step 1 - find\_documents()

```
def find_documents(api=None):
    api = api or CSEApi()
    found = []

    circulars = api.circulars()
    for _, row in circulars.iterrows():
        found.append({
            "Title": str(row.get("fileText", "Untitled")).strip(),
            "Date": row.get("uploadedDate", ""),
            "url": row.get("pdf_url"),
            "id": row.get("id"),
        })

    df = pd.DataFrame(found)

    if not df.empty:
        df = df.drop_duplicates(subset=["url"]).reset_index(drop=True)

    api.scrapers.log.add(f"Found {len(df)} documents to download")
    return df
```

1. `api.circulars()` - Pasindu's method asks CSE for the list of circulars. It has already joined each path onto `https://cdn.cse.lk/cmt/`, so each row has a full `pdf_url`.
2. The loop takes the four things we need from each row: the title, the date, the address and the id number.
3. `pd.DataFrame(found)` - turn the list into a table.
4. `drop_duplicates(subset=["url"])` - if the same address appears twice, keep only one. Downloading a document twice would waste CSE's bandwidth for no benefit.

`for _, row in ... - iterrows()` gives back two things for each row: its row number and the row itself. We do not need the number, so we name it `_`, which is the usual Python way of saying 'I am ignoring this'.

**Remember:** nothing is downloaded in step 1. It only builds the list of what exists and where it lives - the link discovery that makes this crawling.

## Step 2 - download\_documents()

```
wanted = documents.head(limit)
results = []

for number, (_, row) in enumerate(wanted.iterrows(), start=1):
    ...
    title = row["Title"]
    filename = f"{row.get('id', number)}_{title[:60]}"
```

```
# Skip anything we already have.
existing = list(config.PDF_DIR.glob(f"{row.get('id', '')}_*.pdf"))
if existing and str(row.get("id", "")):
    results.append({... "Result": "Already saved" ...})
    continue

reply = scraper.fetch(row["url"], want="bytes")
if not reply["ok"]:
    results.append({... "Result": f"Failed: {reply['error']}" ...})
    continue

content = reply["data"]
if not content[:4] == b"%PDF":
    results.append({... "Result": "Not a PDF file" ...})
    continue

path = save_pdf(content, filename)
results.append({... "Result": "Downloaded" ...})
```

For each wanted document, four checks happen in order:

| Check                  | The code                                                                                         | If it fails                  |
|------------------------|--------------------------------------------------------------------------------------------------|------------------------------|
| Do we already have it? | <code>config.PDF_DIR.glob("14364_*.pdf")</code> looks for a saved file starting with this id     | Skip it - 'Already saved'    |
| Did the download work? | <code>scraper.fetch(url, want="bytes")</code> - through Nuhan's fetch, so it waits and is logged | Record the error and move on |
| Is it really a PDF?    | <code>content[:4] == b"%PDF"</code>                                                              | Do not save it               |
| All good               | <code>save_pdf(content, filename)</code> - Salaama's function writes the file                    | -                            |

- `glob("14364_*.pdf")` - the \* means 'anything here'. It finds any file whose name starts with 14364\_ and ends with .pdf.
- `want="bytes"` - ask `fetch()` for the raw file contents, not text or JSON.
- Each check ends with `continue`, so a problem with one document never stops the others.

`crawl()` at the bottom of the file simply runs step 1 and then step 2.

## What you see when you run it

**Try it yourself.** run your file on its own:

```
python -m src.crawler
```

This is a real run (titles shortened to fit the page):

```
Step 2 - downloading them
Title                                     Result      File
DFCC BANK PLC ("BANK") - BASEL III ...  Already saved  14475_DFCC BANK PLC ...pdf
PAN ASIA BANKING CORPORATION PLC ...    Already saved  14471_PAN ASIA BANKING ...pdf
PEOPLE'S LEASING & FINANCE PLC ...      Already saved  14465_PEOPLES LEASING ...pdf
AITKEN SPENCE HOTEL HOLDINGS PLC ...    Already saved  14457_AITKEN SPENCE ...pdf
DFCC BANK PLC ("BANK") - ISSUANCE ...    Already saved  14436_DFCC BANK PLC ...pdf
```

```
18:23:46 Read robots.txt from the website
18:23:47 POST https://www.cse.lk/api/circularAnnouncement - status 200
18:23:47 Got 5 circulars with PDFs
18:23:47 Found 5 documents to download
```

Every row says 'Already saved' because these five had been downloaded on an earlier run. The whole run sent CSE just **one** request - the list itself - and downloaded nothing twice. The first time a new circular appears, its row says 'Downloaded' instead, and the log shows a GET line for it at least a second and a half after the one before.

# Your files: the Scrapy spider

The same crawling job done a second way, with the Scrapy framework the lecture taught.

src/crawler.py is what the app uses. The spider shows the framework approach used in industry. It runs as a separate program, because Scrapy's engine cannot be started twice inside one running program such as our app.

## The folders

|                                  |                                     |
|----------------------------------|-------------------------------------|
| scrapy_project/<br>scrapy.cfg    | tells Scrapy where the settings are |
| cse_crawler/<br>settings.py      | the rules                           |
| items.py                         | the shape of one record             |
| spiders/<br>cse_announcements.py | the spider itself                   |

This is the same layout that scrapy startproject creates, and the same as the quotes\_crawler example in the course repository.

## The spider, step by step

The spider has three functions. Scrapy calls them in this order:

|                  |                                           |
|------------------|-------------------------------------------|
| start()          | ask CSE for the list of circulars         |
|                  |                                           |
| parse()          | read the list, work out each PDF address, |
|                  | and FOLLOW it <-- the crawling step       |
|                  |                                           |
| parse_document() | record what was found at the other end    |

### start() - the first requests

```
SOURCES = [
    ("https://www.cse.lk/api/circularAnnouncement",
     "reqCircularAnnouncement", "Circular"),
    ("https://www.cse.lk/api/approvedAnnouncement",
     "approvedAnnouncements", "Announcement"),
]

async def start(self):
    for url, record_name, label in self.SOURCES:
        yield scrapy.Request(
            url=url,
            method="POST",
            callback=self.parse,
            cb_kwargs={"record_name": record_name, "label": label},
            dont_filter=True,
        )
```

- Most spiders just list their start addresses in start\_urls, and Scrapy sends ordinary GET requests. We build the requests ourselves because these CSE addresses only answer **POST**.
- callback=self.parse - 'when this reply arrives, give it to parse()'.
- cb\_kwargs - extra information passed along to parse(), so it knows which list the reply came from.
- yield - hand each request to Scrapy, which queues it.

**Be careful:** the course notes call this method `start_requests()`. That was Scrapy's name for years, but newer versions replaced it with `async def start()`, and in version 2.17 - the one we have - `start_requests()` has been removed. Our first version used the old name. The spider ran, collected 0 items and finished almost at once, because Scrapy never called our code - and nothing warned us. It is a real example of a scraper breaking because something underneath it changed.

## parse() - reading the list and following each link

```
def parse(self, response, record_name, label):
    try:
        reply = json.loads(response.text)
    except json.JSONDecodeError:
        self.logger.warning(f"{response.url} did not return JSON")
        return

    records = reply.get(record_name, []) \
        if isinstance(reply, dict) else []
    self.logger.info(f"{label}: found {len(records)} records")

    seen = set()

    for record in records:
        path = record.get("path")
        if not path or not str(path).lower().endswith(".pdf"):
            continue

        pdf_url = self.CDN_BASE + str(path)
        if pdf_url in seen:
            continue
        seen.add(pdf_url)
        ...

        yield scrapy.Request(
            url=pdf_url,
            callback=self.parse_document,
            cb_kwargs={...},
            method="HEAD",
        )
```

- `json.loads(response.text)` - the reply is JSON, not HTML, so we read it with `json` instead of the CSS selectors the quotes spider used. The idea is the same: take the useful parts out of the reply.
- `endswith(".pdf")` - not every announcement has a document; skip those that do not.
- `seen = set()` - a **set** is like a list that never holds the same thing twice. It remembers which addresses we have already followed.
- `yield scrapy.Request(pdf_url, ...)` - **this is the crawling step**: we found a new address and we follow it.

`method="HEAD"` asks only for the **details** of the file - its type and size - without downloading the file itself. The spider only needs to confirm each document exists; downloading megabytes of PDF to find that out would be wasteful.

## parse\_document() - recording the result

```
def parse_document(self, response, title, label, uploaded):
    raw_size = response.headers.get("Content-Length", b "").decode()
    if raw_size == "0":
```

```

raw_size = ""

item = DocumentItem()
item["title"] = title
item["url"] = response.url
item["source"] = label
item["uploaded"] = uploaded
item["content_type"] = response.headers.get(
    "Content-Type", b"unknown").decode()
item["size_kb"] = (round(int(raw_size) / 1024, 1)
                   if raw_size.isdigit() else None)
item["status"] = response.status
yield item

```

Each finished `item` is one record in the output file. Note the size. Content - Length is the header where a server says how big a file is, and CSE's server does not always fill it in properly - on our run, two of the five PDFs came back saying 0. A real PDF is never empty, so we record None instead. Zero would be a **wrong** measurement - it would say the file is empty. None honestly says 'we did not find out'.

**Be careful:** our first version only handled a missing size, not a size of 0. The output file said two real documents were 0.0 KB, and it was only noticed by reading the output. Always look at what your scraper actually produced - not just whether it ran without errors.

## items.py - the shape of one record

```

class DocumentItem(scrapy.Item):
    title = scrapy.Field()
    url = scrapy.Field()
    source = scrapy.Field()
    uploaded = scrapy.Field()
    content_type = scrapy.Field()
    size_kb = scrapy.Field()
    status = scrapy.Field()

```

Declaring the fields up front is a safety check. If somebody types `item["ttile"]` by mistake, Scrapy raises an error at once instead of quietly creating a misspelled column in the output.

## settings.py - the polite rules

| Setting                  | Value           | What it means                                                                                                             |
|--------------------------|-----------------|---------------------------------------------------------------------------------------------------------------------------|
| ROBOTSTXT_OBEY           | True            | Scrapy reads robots.txt itself and refuses blocked addresses. Many tutorials tell you to switch this off - we left it on. |
| USER_AGENT               | our honest name | The same as the rest of the project                                                                                       |
| DOWNLOAD_DELAY           | 1.5             | Wait 1.5 seconds between requests                                                                                         |
| RANDOMIZE_DOWNLOAD_DELAY | True            | Vary the wait a little, so requests do not arrive in a machine-gun rhythm                                                 |
| CONCURRENT_REQUESTS      | 1               | One request at a time. Normally Scrapy sends up to 8 at once to the same website.                                         |

| Setting               | Value | What it means                                                                          |
|-----------------------|-------|----------------------------------------------------------------------------------------|
| AUTOTHROTTLE_ENABLED  | True  | Slow down automatically if CSE's server starts replying slowly                         |
| CLOSESPIDER_ITEMCOUNT | 40    | Stop after 40 items - enough to show it works                                          |
| HTTPCACHE_ENABLED     | True  | Remember replies for 15 minutes, so running it again soon after does not ask CSE again |

**Remember:** your own crawler gets its politeness from Nuhan's fetch(). The Scrapy spider gets the same politeness from its settings file - robots.txt obeyed, 1.5 seconds between requests, one at a time.

## Running the spider

**Try it yourself.** from the project folder:

```
cd scrapy_project
scrapy crawl cse_announcements -o ../data/scrapy_output.json
```

`crawl cse_announcements` runs the spider by its name. `-o` (a capital O) saves the items to a JSON file, replacing the old one. A small `-o` would **add** to the end of the old file instead, and two JSON lists stuck end to end is no longer valid JSON.

When it finishes, Scrapy prints a summary. These lines are from a real run:

```
[cse_announcements] INFO: Circular: found 5 records
[cse_announcements] INFO: Announcement: found 125 records
'downloader/request_count': 9,
'elapsed_time_seconds': 14.64,
'item_scraped_count': 5,
'robotstxt/request_count': 2,
```

- The circulars list had 5 records, each with a PDF. The announcements list had 125 records, but none had a PDF attached, so the spider skipped them all.
- 9 requests in total: robots.txt from both servers (2), the two lists (2), and one HEAD request for each of the 5 PDFs (5).
- `robotstxt/request_count: 2` - Scrapy read robots.txt from `www.cse.lk` and `cdn.cse.lk` **before** crawling either of them.
- About 15 seconds for 9 requests - that is the delay and the one-at-a-time rule working.

Run it again within 15 minutes and the summary says `'httpcache/hit': 9` - every reply came from the cache, and CSE received no requests at all.

# Your file: src/cleaner.py

Tidies the table of share prices so it can be sorted, totalled and charted - and then checks that the values make sense.

## Part 1 - describe(): counting the problems

```
def describe(df):
    ...
    return {
        "Rows": len(df),
        "Columns": len(df.columns),
        "Empty cells": int(df.isna().sum().sum()),
        "Empty columns": int(df.isna().all().sum()),
        "Duplicate rows": int(df.duplicated().sum()),
        "Number columns": int(df.select_dtypes(include="number").shape[1]),
        "Date columns": int(df.select_dtypes(include="datetime").shape[1]),
    }
```

This counts problems in the table. It runs **before** and **after** cleaning, so the two can be shown side by side. Saying 'we cleaned the data' proves nothing; showing the counts change does.

- `df.isna()` marks every missing cell as True. `.sum().sum()` counts them - once down each column, then across.
- `df.isna().all()` - is a column missing in **every** row?
- `df.duplicated()` - is this row an exact copy of an earlier one?
- `select_dtypes(include="number")` - just the number columns. `.shape[1]` is how many columns that is.

## Part 2 - clean(): six steps

```
def clean(df):
    ...
    tidy = df.copy()
    changes = []
```

`df.copy()` first, always. The original table stays untouched, so if the cleaning ever turns out to be wrong, we can compare with the original - without downloading everything from CSE again. `changes` collects a sentence about each thing we did.

### Step 1 - turning big numbers back into dates

```
for column in tidy.columns:
    if not any(word in column.lower()
               for word in ("date", "time", "created")):
        continue
    if not pd.api.types.is_numeric_dtype(tidy[column]):
        continue

    values = tidy[column].dropna()
    if not values.empty and values.median() > 1_000_000_000_000:
        tidy[column] = (pd.to_datetime(tidy[column], unit="ms",
                                       utc=True, errors="coerce")
                       .dt.tz_convert("Asia/Colombo")
                       .dt.tz_localize(None))
```

```
date_columns.append(column)
```

A column is converted only if it passes **three** tests:

1. Its name contains 'date', 'time' or 'created'.
2. It holds numbers.
3. Its middle value (the **median**) is over one trillion (1\_000\_000\_000\_000 - the underscores are only there to make it readable). Any recent date in milliseconds is about 1.8 trillion, but no price or quantity ever is.

The three tests together mean we never turn a price into a date by accident, just because its column happens to have 'date' in the name. `unit="ms"` tells pandas the numbers are milliseconds.

`errors="coerce"` means any value that cannot be converted becomes missing instead of stopping everything.

The last three lines deal with **time zones**:

- The milliseconds are counted in **UTC**, the world's standard time. `utc=True` tells pandas so.
- Sri Lanka is 5 hours 30 minutes ahead of UTC. `tz_convert("Asia/Colombo")` moves every time into Colombo time.
- `tz_localize(None)` removes the time zone label afterwards, because Excel cannot save a date that carries one.

**Be careful:** our first version left out the time zone. The table looked fine - real dates, real times - but every trade showed between 04:00 and 09:00. The CSE only trades from 9.30 in the morning to 2.30 in the afternoon, so every time was 5 hours 30 minutes early. After the fix they run from 09:30 to 14:30. Nothing crashed and nothing warned us: it was found by asking whether the values made sense, which is exactly what cleaning and checking are for.

## Step 2 - turning text that is really numbers into numbers

```
for column in tidy.select_dtypes(include=["object", "str"]).columns:
    sample = tidy[column].dropna().astype(str).head(50)
    ...
    converted = pd.to_numeric(sample.str.replace(",", "",
                                                regex=False),
                              errors="coerce")
    if converted.notna().mean() > 0.9:
        tidy[column] = pd.to_numeric(...)
        number_columns.append(column)
```

- `select_dtypes(include=["object", "str"])` - just the text columns. Newer pandas keeps text in its own `str` type and older pandas used `object`, so we ask for both.
- Take a sample of 50 values and try turning them into numbers. `.str.replace(",", "", regex=False)` removes commas first, so '1,094.25' becomes '1094.25'.
- `converted.notna().mean()` is the share that converted - `.mean()` of True and False values gives the fraction that are True.
- If more than 90% convert, it was a number column dressed as text, and the whole column is converted. Not 100%, because a real number column can still have a few blanks or dashes.

## Step 3 - trimming extra spaces

```
for column in tidy.select_dtypes(include=["object", "str"]).columns:
    tidy[column] = tidy[column].apply(
        lambda value: value.strip() if isinstance(value, str) else value)
```

" SAMPATH BANK " and "SAMPATH BANK" look the same to a person but are different to a computer - grouping by name would give two groups. `.strip()` removes spaces from both ends.

Only real text is trimmed. A missing value is left missing. That matters: an older way of writing this turned missing values into the text 'nan', which hides gaps and makes the table look more complete than it really is.

## Steps 4, 5 and 6

```
# 4. Drop columns that are empty for every row
empty = [c for c in tidy.columns if tidy[c].isna().all()]
if empty:
    tidy = tidy.drop(columns=empty)

# 5. Remove repeated rows
tidy = tidy.drop_duplicates().reset_index(drop=True)

# 6. Put the useful columns first
useful = ["symbol", "name", "price", "change",
          "percentageChange", "quantity", "turnover"]
front = [c for c in useful if c in tidy.columns]
tidy = tidy[front + [c for c in tidy.columns if c not in front]]
```

- **Step 4** - a column that is empty for every company is only clutter.
- **Step 5** - repeated rows would count the same company twice.
- **Step 6** - purely for people: the table opens with the symbol, name and price, instead of internal id numbers.

## Part 3 - check(): do the values make sense?

```
if "price" in df.columns:
    negative = int((df["price"] < 0).sum())
    add("No negative prices", negative == 0, ...)

if "percentageChange" in df.columns:
    extreme = int((df["percentageChange"].abs() > 100).sum())
    add("Price changes look sensible", extreme == 0, ...)
```

`df["price"] < 0` marks every negative price as True, and `.sum()` counts them. `.abs()` means 'ignore the minus sign', so it catches moves of more than 100% up or down.

| Check                       | Why                                   |
|-----------------------------|---------------------------------------|
| Rows collected              | Is there any data at all?             |
| No negative prices          | A share cannot cost less than nothing |
| Price changes look sensible | More than 100% in a day is suspicious |
| Every row has a symbol      | Every company needs its code          |
| No company appears twice    | Each symbol should be there once      |
| How complete is the data    | What share of cells have a value      |

A check that fails says '**Check this**' - it does not delete anything. A big price move can be real, after a share split for example. Automatically deleting rows that look odd is how real data gets quietly lost.

## What you see when you run it

**Try it yourself.** run your file on its own:

```
python -m src.cleaner
```

This is a real run:

What we changed

- Turned 1 column(s) of large numbers back into dates: lastTradedTime
- Removed extra spaces from the text columns
- Moved the most useful columns to the front

Before and after

| Check          | Before | After |
|----------------|--------|-------|
| Rows           | 284    | 284   |
| Columns        | 23     | 23    |
| Empty cells    | 14     | 14    |
| Empty columns  | 0      | 0     |
| Duplicate rows | 0      | 0     |
| Number columns | 19     | 18    |
| Date columns   | 0      | 1     |

Checks

|                             | Check              | Result | Detail                       |
|-----------------------------|--------------------|--------|------------------------------|
|                             | Rows collected     | OK     | 284 rows                     |
|                             | No negative prices | OK     | All prices are zero or above |
| Price changes look sensible |                    | OK     | Nothing moved more than 100% |
| Every row has a symbol      |                    | OK     | All rows have one            |
| No company appears twice    |                    | OK     | Each company appears once    |
| How complete is the data    |                    | OK     | 99.8% of cells have a value  |

Look at 'Number columns 19 -> 18' and 'Date columns 0 -> 1'. That is one column - lastTradedTime - moving from being counted as a number to being counted as a date. The two changes together are the fix showing up in the counts.

Some steps changed nothing on this run - there were no empty columns and no repeated rows, and no text columns needed turning into numbers. That is fine. The steps are there for the days the data does arrive with those problems, and `describe()` proves whether they were needed. The number of companies changes from day to day - usually somewhere around 284 to 291.

**Remember:** cleaning fixes the shape - dates, number types, spaces, empty columns, repeats. Checking asks whether the values are believable. And the original is always kept, untouched.

# How your part fits into the whole project

Every button in the app follows the same path. The lines marked <-- YOU are yours.

|                                        |                      |         |
|----------------------------------------|----------------------|---------|
| 1. You click a button in the app       | app.py               |         |
| 2. The app asks for some data          | src/api_client.py    |         |
| 3. Every request goes through fetch()  | src/ethics.py        |         |
| a. check robots.txt                    | src/ethics.py        |         |
| b. wait 1.5 seconds                    | src/ethics.py        |         |
| c. send it, saying who we are          | src/ethics.py        |         |
| d. write it in the request log         | src/ethics.py        |         |
| 4. The reply (JSON) becomes a table    | src/api_client.py    |         |
| 5. Documents are found and downloaded  | src/crawler.py       | <-- YOU |
| 6. Their text is read                  | src/pdf_extractor.py |         |
| 7. Scanned pages are read by OCR       | src/ocr_extractor.py |         |
| 8. The table is tidied and checked     | src/cleaner.py       | <-- YOU |
| 9. Charts are drawn                    | src/analysis.py      |         |
| 10. Files are saved (CSV, Excel, JSON) | src/storage.py       |         |

Two more pieces sit beside this path rather than on it: `src/static_scraper.py` shows the ordinary requests-and-BeautifulSoup method, and `src/selenium_scraper.py` shows the real-browser method. The Scrapy spider in `scrapy_project/` does the document search a second way, with the Scrapy framework.

Your part is step 5 and step 8. At step 5, your crawler takes the list of circulars from Pasindu's code and downloads the documents - which Nuhan's PDF reader and Salaama's OCR then read. At step 8, your cleaner tidies the share price table from Pasindu's code before Salaama's charts are drawn from it and her code saves it to files. Both of your steps sit in the middle of the project, joining the collecting to the using.

# Ethics - what every member must know

The assignment says marks are given for **following ethical scraping practices** and for **being able to explain your work**. Ethics was Nuhan's file, but any of us can be asked about it, so here is the short version.

## Ethical and unethical scraping

| Ethical (what we do)                                | Unethical (what we avoid)                        |
|-----------------------------------------------------|--------------------------------------------------|
| Only collect what anyone can see without logging in | Getting past logins, paywalls or CAPTCHAs        |
| Follow the website's robots.txt                     | Ignoring robots.txt                              |
| Wait between requests                               | Firing requests as fast as possible              |
| Collect no personal information                     | Collecting data about people                     |
| Credit the source                                   | Using the data without saying where it came from |

## robots.txt

**robots.txt** is a small file a website publishes to tell programs like ours which parts they may visit. CSE's says:

```
User-agent: *  
Disallow:  
Disallow: /cgi-bin/  
Sitemap: https://www.cse.lk/sitemap.xml
```

User-agent: \* means the rules are for every program. The next two lines ask programs to keep out of one folder, /cgi-bin/. None of the addresses we use are in that folder. There is no line asking us to wait between requests.

**Think of it like this.** robots.txt is a sign on a shop door that says 'Staff only beyond this point'. Nothing physically stops you walking past it - you stay out because you respect it.

## What our code does about it

1. Every request goes through one function, `fetch()` in `src/ethics.py`. It checks robots.txt first.
2. It waits **1.5 seconds** between requests, even though CSE does not ask us to.
3. It sends a **User-Agent** with our university, course and email.
4. It writes every request to `logs/request_log.csv`.
5. It saves a copy of each reply so we never ask twice.
6. Every exported file names the **Colombo Stock Exchange** as the source.

**Remember:** 'We only collect public data, we follow robots.txt, we wait 1.5 seconds between requests, we say who we are, and we credit CSE in every file we save.' If you are asked about ethics, start with that sentence.

# Showing your part in the viva

The viva is 8 minutes for the whole team: about 3 minutes of presentation, then 5 minutes of questions about what each member did. This chapter gets you ready for your share of it.

## Before the viva

1. Double-click `Run_CSE_App.bat` in the project folder. The app opens in your browser. Keep the black window open - closing it stops the app.
2. Check your tab works, as below.
3. Run your files once from the terminal, so you have seen the output yourself.

## Showing it in the app

1. Open the **Announcements** tab. Your name is on the label at the top.
2. Press **Get the announcements** - the list of company notices.
3. Under **Download the documents**, leave the number at 5 and press **Download**. Point to the two tables: the documents found, and what happened to each - 'Downloaded' or 'Already saved'.
4. Open the **Data & Export** tab (after the market data has been collected on the Market Data tab). Press **Clean and check**. Point to the before-and-after table and the checks.

## Showing it from the terminal

```
python -m src.crawler          # find and download PDFs
python -m src.cleaner          # before/after and checks
cd scrapy_project
scrapy crawl cse_announcements -O ../data/scrapy_output.json
```

## Your 30-second explanation

Practise saying this out loud until it feels natural. Change the words so they sound like you.

"My part is crawling and cleaning. Scraping takes data from a page you know; crawling finds pages by following links. We did not know which documents CSE had published, so my crawler reads the list of circulars, builds each PDF's full address - they live on a separate server, `cdn.cse.lk` - and downloads them one at a time, skipping duplicates and checking each file really starts with `percent-PDF`. I did the same job again as a Scrapy spider, with `robots.txt` obeyed and a 1.5 second delay in its settings. Then I clean the share price table: dates that arrive as huge numbers become real dates, text that is really numbers becomes numbers, and I check the values make sense - for example, no negative prices."

## If the examiner points at your code

These are the parts of your files most likely to be pointed at, and what to say about each:

| If they point at...                          | Say...                                                                                   |
|----------------------------------------------|------------------------------------------------------------------------------------------|
| <code>drop_duplicates(subset=["url"])</code> | The same document can be listed twice; downloading it twice would waste CSE's bandwidth. |

| If they point at...                                        | Say...                                                                                       |
|------------------------------------------------------------|----------------------------------------------------------------------------------------------|
| <code>content[:4] == b"%PDF"</code>                        | Every PDF starts with %PDF. If not, we got something else, like an error page.               |
| <code>scraper.fetch(row["url"], want="bytes")</code>       | Downloads go through Nuhan's fetch, so robots.txt, the delay and the log apply.              |
| <code>yield scrapy.Request(pdf_url, ...) in parse()</code> | The crawling step - we found a new address and follow it.                                    |
| <code>async def start(self)</code>                         | Scrapy 2.17 removed start_requests; start() is its replacement.                              |
| <code>ROBOTSTXT_OBEY = True</code>                         | Scrapy checks robots.txt itself. We left it on.                                              |
| <code>values.median() &gt; 1_000_000_000_000</code>        | Only numbers that big can be dates in milliseconds - so we never convert a price by mistake. |
| <code>tidy = df.copy()</code>                              | The original is never changed, so we can always compare against it.                          |

# Practice questions

These are the same 50 questions as your section of the team guide: about 30 on your own part, then 20 on the project as a whole. Cover each answer with your hand and try to say it first. By now, every answer should make sense - if one does not, go back to the chapter it comes from.

## EASY - THE BASICS

### 1. Which files did you write?

`src/crawler.py`, which finds and downloads the PDF documents; `src/cleaner.py`, which tidies the data and checks it; and the Scrapy project in `scrapy_project/`.

### 2. What is the difference between scraping and crawling?

Scraping is taking data from a page you already know about. Crawling is finding pages by following links from one to the next.

Most of our project is scraping, because we know the address. My part is the crawling - we do not know which documents exist until we look.

### 3. Why are crawlers sometimes called spiders?

Because they crawl across the web the way a spider moves across its web. They start from a few known addresses and follow links outward, building up a network of pages they have found.

### 4. What is Scrapy?

A Python framework built specifically for crawling and scraping. It handles the queue of requests, the delays, `robots.txt` and saving the output, so you only write the part that says what to extract and which links to follow.

### 5. Why does scraped data need cleaning at all?

Because it comes out in whatever shape the website happened to send it, not in a shape that is ready to analyse. The course listed 'data collected through scraping is ready for analysis' as a misconception.

### 6. What are the four things you had to clean in the CSE data?

Dates arriving as very large numbers. Numbers arriving as text. Columns that are empty for every single company. And text with extra spaces around it.

### 7. Why does the crawler skip documents you already have?

Because downloading the same file twice costs CSE bandwidth and gains us nothing. Before each download we look in `data/pdfs/` for a file already saved under that document id, and if it is there we move on.

### 8. What is web scraping?

Collecting data from websites automatically with a program, instead of copying it by hand. You send a request for a page, get the content back, find the parts you want, and save them in a structured form such as a CSV file.

### 9. What website did your group work on, and what did you collect?

The Colombo Stock Exchange, `www.cse.lk`. We collect share prices for around 290 listed companies, details for individual companies such as market value and yearly high and low, and company announcements together with the PDF documents attached to them.

## 10. Is the data you collected public?

Yes. Everything we collect can be seen by anybody visiting the site without logging in. CSE publishes it so the investing public can read it. We collect no personal information about any individual - only company figures.

## 11. What is robots.txt?

A text file websites publish at the top level of the site telling automated programs which pages they may and may not visit. It is a voluntary guideline, not a lock - a program can ignore it. Ethical scrapers choose to follow it.

## 12. What does CSE's robots.txt say?

It blocks one section, /cgi-bin/, and allows everything else. It does not set a Crawl-delay, so it does not ask us to wait between requests at all.

## 13. If the site does not ask you to wait, why do you wait?

Because being allowed to go fast is not a reason to. Sending requests as fast as the computer can manage would put load on their server and gain us nothing. We wait 1.5 seconds between requests.

## 14. What does the HTTP status code 200 mean?

The request worked. 403 means we are not allowed, 404 means the page does not exist, and 500 means the server had a problem.

## 15. What is a DataFrame?

A table in pandas, with rows and named columns, a bit like a sheet in Excel. Almost everything we collect ends up in one, because it makes the data easy to sort, filter and save.

## MEDIUM - HOW IT WORKS

## 16. Walk me through your crawler.

Two stages.

`find_documents()` asks for the list of circulars, and for each one builds the full address of the PDF. It also removes duplicates, because the same document can appear twice and downloading it twice would waste CSE's bandwidth for nothing.

`download_documents()` then fetches them one at a time through the ethical session, so each download waits its 1.5 seconds. It skips anything we already have on disk.

## 17. How did you work out where the PDFs are stored?

The list only gives part of the address, something like `upload_report_file/abc123.pdf`, which is not a working link on its own. And the PDFs are not on the website itself - they are on a separate server.

So we tried the likely addresses until one returned a real PDF. `www.cse.lk/cmt/...` gave 404, `cdn.cse.lk/...` gave 403, and `cdn.cse.lk/cmt/...` returned 200 with content type `application/pdf`.

## 18. How do you check a downloaded file really is a PDF?

Every PDF file starts with the four characters `%PDF`. We check the first four bytes before saving. If they do not match, something else came back - an error page, for instance - and we do not save it as a PDF.

```
if not content[:4] == b"%PDF":
    results.append({"Title": title, "Result": "Not a PDF file"})
    continue
```

## 19. Why limit how many documents you download?

There are far more documents on the site than we need to show the technique works. Downloading all of them just because we could would put load on their server for no benefit. The lecture put it as: only scrape the data you need.

## 20. Explain the Scrapy spider.

`start()` sends POST requests to the two announcement addresses, because CSE does not answer GET. `parse()` reads the JSON reply, builds the full address of each PDF, and yields a new request for it - that yielding of a new request is the crawling step. `parse_document()` then records what came back at the other end.

```
def parse(self, response, record_name, label):
    reply = json.loads(response.text)
    for record in reply.get(record_name, []):
        pdf_url = self.CDN_BASE + record["path"]
        yield scrapy.Request(pdf_url,                # follow the link
                             callback=self.parse_document,
                             method="HEAD")
```

## 21. Why does the spider use HEAD instead of GET?

A HEAD request asks only for the details of the file - its type and its size - without downloading the file itself. We want to confirm each document exists and see how big it is, and downloading several megabytes of PDF just to find that out would be wasteful.

## 22. Which Scrapy settings keep the spider polite?

`ROBOTSTXT_OBEY = True`, which makes Scrapy check robots.txt itself. `DOWNLOAD_DELAY = 1.5`, matching the rest of our project. `CONCURRENT_REQUESTS = 1`, where the default is 8 - that would mean eight connections at once. `AutoThrottle`, which slows down further if the server starts responding slowly. And `CLOSESPIDER_ITEMCOUNT = 40` so it stops after enough items.

## 23. Why does the spider record None rather than 0 when the file size is missing?

Because 0 is a measurement, and a wrong one - it says the file is empty when it is not. None honestly says we did not find out.

The CSE server does not always send the Content-Length header back on a HEAD request. Writing 0 in that case would put false figures into our dataset, which is exactly the kind of quiet data quality problem the course warned about.

## 24. Explain the date problem.

CSE sends dates as the number of milliseconds since 1 January 1970 - a number like 1786440420412. Left alone, pandas treats that as an ordinary number, which means you could sort by it, or worse, calculate an average of it, and nothing would warn you.

We convert those columns with `pd.to_datetime(column, unit="ms", utc=True)`, and then into Colombo time. The numbers are counted in UTC, and Sri Lanka is 5 hours 30 minutes ahead - without that step every trade would look as if it happened before the market opened.

## 25. How do you avoid converting a column by mistake?

Three checks, and all of them have to pass. The column name has to contain a word like 'date', 'time' or 'created'. The column has to hold numbers. And the values themselves have to be larger than a trillion, which any real date in milliseconds will be but a price or a quantity never is.

```
if not any(word in column.lower()
           for word in ("date", "time", "created")):
    continue
if not pd.api.types.is_numeric_dtype(tidy[column]):
    continue
...
if not values.empty and values.median() > 1_000_000_000_000:
    tidy[column] = (pd.to_datetime(tidy[column], unit="ms", utc=True,
                                  errors="coerce")
                   .dt.tz_convert("Asia/Colombo")
                   .dt.tz_localize(None))
```

## 26. Why does a number stored as text matter?

Because you cannot add up text, and sorting it sorts alphabetically. That puts "9" after "100", because it compares the first character. Any chart or total built on it would be wrong.

## 27. How do you decide a text column is really numbers?

We take a sample of the column, try converting it to numbers, and see how much of it succeeds. If more than 90 per cent converts, it was a number column stored as text and we convert the whole thing.

We do not require 100 per cent, because a genuine number column can still have a few blanks or dashes in it.

## 28. What is the difference between cleaning and checking?

Cleaning fixes the shape of the data - the types, the empty columns, the duplicates. Checking asks whether the values are believable.

A share price of minus five rupees would be perfectly tidy and completely wrong. Cleaning would never catch it; a check does.

## 29. What checks do you run?

That there are rows at all. That no price is negative. That nothing moved by more than 100 per cent in a day, which would be suspicious. That every row has a trading symbol. That no company appears twice. And an overall completeness figure - what percentage of cells actually have a value.

## 30. Why do you copy the DataFrame before cleaning it?

So the original stays untouched. If the cleaning turns out to have a bug, we can look at the raw version and compare, without having to go back to CSE and download everything again.

## 31. How do you show that the cleaning did something?

We count the problems before and after and show them side by side - the number of rows, empty cells, empty columns, duplicate rows, number columns and date columns. Saying 'we cleaned the data' means very little; showing date columns going from 0 to 1 is evidence.

## 32. Why did requests and BeautifulSoup not work on the CSE site?

Because the page is dynamic. requests downloads the HTML file and stops - it does not run JavaScript. The CSE server sends an almost empty page, and a program inside the browser fetches the prices and puts them on the page afterwards. So the file we download has about 24 characters of text in it and no tables.

### 33. What is the difference between static and dynamic content?

On a static page the content is already inside the HTML file the server sends, so downloading the file is enough. On a dynamic page the file arrives nearly empty and JavaScript fills it in afterwards, so you need either a real browser or the address the page gets its data from.

### 34. How did you find the addresses the site uses for its data?

We opened the CSE site in a browser, pressed F12 to open the developer tools, went to the Network tab, and reloaded the page. That tab lists every request the page makes, and it showed the page fetching its own data from addresses like `/api/tradeSummary`.

### 35. What is JSON, and why is it convenient here?

JSON is a structured text format for data, made of names and values. It is convenient because it is already organised - pandas can turn it straight into a table, with no need to hunt through HTML tags for the numbers.

### 36. How does your project make sure the ethical rules are followed?

Every request in the whole project goes through one function, `Scraper.fetch()` in `src/ethics.py`. That function checks `robots.txt`, waits the delay, sends the request with our User-Agent, and writes a line to the log. Because there is only one way out to the internet, the rules cannot be skipped by accident in some other file.

### 37. What does your User-Agent say, and why does that matter?

It gives the university, the course, the group and an email address, instead of pretending to be an ordinary browser. If our requests ever caused a problem, the site administrator could see who was responsible and get in touch.

### 38. How much load did you put on CSE's servers?

Around 30 to 40 requests for a full run of the app, spaced at least 1.5 seconds apart. Every one is recorded in `logs/request_log.csv`. Somebody browsing the same pages by hand would actually generate more traffic, because each page load also pulls scripts, fonts and images that we never request.

### 39. Why are there two kinds of PDF in the project?

A text based PDF was made by a computer, so the letters inside it are real text and we can read them straight out. A scanned PDF is a photograph of a printed page - the words are part of a picture, so there is no text inside to extract and it has to go through OCR instead.

## HARDER - EXPLAIN AND DEFEND IT

### 40. Your first version of the spider collected nothing. What happened?

It reported 0 items and finished in about a hundredth of a second, without sending a single request.

The method for sending the first requests used to be called `start_requests()`, which is what the course notes use. Scrapy 2.13 replaced it with an asynchronous `start()` method, and by version 2.17, which is what we have installed, `start_requests()` was removed completely. So Scrapy never called our method and nothing warned us.

It is a real example of a scraper breaking because something underneath it changed - usually that is the website, but here it was the library.

### 41. Why does the project have two crawlers doing the same job?

`src/crawler.py` is ours, written directly with `requests`, and it is what the app actually uses. It is simple and it goes through the same ethical layer as everything else in the project.

The Scrapy version shows the framework approach that the course taught and that industry uses. It runs as a separate process, because Scrapy's engine cannot be started twice inside one program.

#### **42. How would you extend the crawler to follow links deeper?**

At the moment we go two levels: the listing, then each document. To go deeper you would have `parse_document()` yield further requests for any links it found, and Scrapy would queue those the same way.

You would also need to keep a set of addresses already visited so it cannot loop forever, and set `DEPTH_LIMIT` so it does not wander across the whole site. `allowed_domains` already stops it leaving `cse.lk`.

#### **43. Your before and after table shows the number columns going down by one. Why is that a good thing?**

Because that column became a date column instead. It was being counted as a number, which is exactly the problem - it was a timestamp being treated as an ordinary figure.

So one column moved out of the number count and into the date count. The two changes together are the fix showing up in the numbers.

#### **44. Is dropping empty columns always the right thing to do?**

Not always, and it is worth being honest about. A column that is empty today might be filled in tomorrow, or might be empty only for the particular set of companies we happened to fetch.

We do it because those columns are noise on screen and in every exported file, and because we keep the raw version, so nothing is actually lost. If we were building something that ran every day, dropping columns based on one day's data would be the wrong call.

#### **45. How would you handle a company that legitimately moved more than 100 per cent?**

The check does not delete anything - it flags it. That is the important part of the design.

A big move can be real, after a share split or a return from suspension. So the check says 'look at this', not 'this is wrong'. Automatically deleting rows that look odd is how you quietly lose real data.

#### **46. If you had more time, what would you add to the cleaning?**

Two things. Cross-checking against another source, so we could tell whether a strange number is a collection error or a real market event. And keeping the data from several days, which would let us spot a value that is impossible given yesterday's - a price that has jumped a hundred times overnight, for example.

#### **47. Is web scraping legal?**

It is not automatically legal or illegal - it depends on the data, the website's terms, and what you do with the result.

We stayed well on the safe side: only public company information, no personal data, `robots.txt` followed, no security measure bypassed, the source credited in every file, and academic use only. We also did not ask CSE for written permission, which would be the right next step if we wanted to publish or sell anything based on this.

#### **48. Is it acceptable to use an address that is not officially documented?**

We think so, for three reasons.

It needs no login and no key, so it is exactly the same request the page makes for any visitor. `robots.txt` does not ask us to stay away from it. And it is actually less work for their server than loading the whole page, because loading the page makes them send the HTML, the JavaScript, the fonts and the images, and the browser then requests that same data address at the end of it anyway.

The honest limitation is that an undocumented address can change without warning, and that is CSE's right.

**49. Did you get past any CAPTCHAs or logins?**

No, and we would not. Everything we collect is visible without logging in, and we never met a CAPTCHA because we never behaved in a way that would set one off.

A CAPTCHA is a website asking whether you are a person. Getting around one means answering yes when the honest answer is no. Being able to defeat a control is not the same as being allowed to.

**50. What would break this project, and how bad would it be?**

CSE changing the addresses their site uses to load data. They are not officially documented, so they could change at any time without notice.

It would not be a disaster. Every address is listed in one place, in `config.py`, so fixing it means editing that table rather than hunting through the whole project. We also save a copy of every reply, so a demonstration still works from the saved data even if the live site has changed.

# Word list

Every special word used in this document, from A to Z.

| Word          | What it means                                                                   |
|---------------|---------------------------------------------------------------------------------|
| Bytes         | The raw contents of a file, as small numbers from 0 to 255.                     |
| Callback      | The function Scrapy calls when a reply arrives.                                 |
| CDN           | Content Delivery Network - a server built just for delivering files.            |
| Cleaning      | Fixing the shape of data - types, gaps, repeats, spaces.                        |
| Crawling      | Finding pages or files by following links.                                      |
| CSV           | A simple table file: one row per line, commas between columns.                  |
| DataFrame     | A table in pandas, with rows and named columns.                                 |
| Duplicate     | A row or item that is an exact copy of another.                                 |
| Dynamic page  | A page whose content is added by JavaScript later.                              |
| Framework     | A tool that runs the show and calls your code at the right moments.             |
| Function      | A named set of steps that can be called to do a job.                            |
| HEAD request  | A request for a file's details without the file itself.                         |
| HTML          | The language web pages are written in, using tags.                              |
| Inheritance   | Building a class on top of another, so it gets everything the other can do.     |
| Item          | One finished record produced by a Scrapy spider.                                |
| JavaScript    | A language that runs inside the browser and can change the page after it loads. |
| JSON          | A tidy text format for data: names and values.                                  |
| Library       | Code somebody else wrote that we can import and use.                            |
| Median        | The middle value when all the values are put in order.                          |
| Missing value | A gap in the data - shown as None or NaN.                                       |
| PDF           | A document file that looks the same on every computer.                          |
| Request       | A message asking a server for something.                                        |
| Response      | The server's reply to a request.                                                |
| robots.txt    | A file where a website says which parts programs may visit.                     |
| Scrapy        | A Python framework built for crawling.                                          |
| Server        | A computer that stores a website and answers requests.                          |
| Set           | A collection that never holds the same thing twice.                             |
| Share         | A small piece of a company that people can buy and sell.                        |
| Spider        | A crawler - in Scrapy, the class that does the crawling.                        |

| Word               | What it means                                              |
|--------------------|------------------------------------------------------------|
| <b>Static page</b> | A page whose content is already in the HTML file.          |
| <b>Status code</b> | A number on each reply saying how it went - 200 means OK.  |
| <b>Tag</b>         | A label in HTML, such as <title> or <table>.               |
| <b>Type</b>        | What kind of value something is: number, text, date.       |
| <b>URL</b>         | The address of a page or file on the web.                  |
| <b>User-Agent</b>  | The part of a request that says which program sent it.     |
| <b>Variable</b>    | A name that holds a value.                                 |
| <b>yield</b>       | Hand something back and carry on, ready to hand back more. |